



DEEP LEARNING AND NATURAL LANGUAGE PROCESSING

LECTURE 7: STATISTICAL NLP AND WORD EMBEDDINGS

Dr. Adewale Akinfaderin

SEAS 8520-D6A · Summer 2026



Table of Contents

1 Recap and Roadmap

- ▶ Recap and Roadmap
- ▶ NLP: The Bird's-Eye View
- ▶ Exploring the IMDB Data
- ▶ Preparing Text Data
- ▶ Bag-of-Words on IMDB
- ▶ Word Embeddings
- ▶ Wrap-Up



Recap: From Grids to Sequences

1 Recap and Roadmap

Last time (CNNs)

- Convolution builds in **locality** and **weight sharing** for grid data.
- A small CNN beat a dense MLP on images with *fewer* parameters.

Today (text)

- Images are fixed numeric grids. **Text** is variable-length symbols with no built-in numbers.
- Two questions drive the lecture: how do we turn words into vectors, and does **word order** matter?

1. **Explore the IMDB data:** class balance, review length, frequent n-grams.
2. **Prepare text data:** standardize, tokenize, index, and subword (BPE) tokenization.
3. **Bag-of-words on IMDB:** unigrams, bigrams, and TF-IDF, on one shared classifier.
4. **Word embeddings:** the distributional hypothesis, cosine similarity, and word2vec.

The companion notebook runs the full pipeline; every number and plot on these slides comes from that run.



Table of Contents

2 NLP: The Bird's-Eye View

- ▶ Recap and Roadmap
- ▶ **NLP: The Bird's-Eye View**
- ▶ Exploring the IMDB Data
- ▶ Preparing Text Data
- ▶ Bag-of-Words on IMDB
- ▶ Word Embeddings
- ▶ Wrap-Up

Modern NLP does not try to *understand* language; it ingests text and returns something useful from statistical regularities. Natural language is **usage-first** (rules formalized after the fact and routinely broken), so decades of hand-written rules plateaued. It helps to group tasks by the **shape** of their output, not the topic:

Family	Output	Examples
Sequence classification	one label per text	sentiment, topic, spam
Sequence labeling	one label per token	POS tagging, NER
Span extraction	a span of the input	extractive QA
Sequence-to-sequence	a new sequence	translation, summarization
Generation / language modeling	next token(s)	autocomplete, chat

Today's task, **sentiment analysis**, is the simplest family: one label per document. Every row starts from the same obstacle: a neural network cannot read raw strings.

Encoding an *individual* word is settled (a categorical feature mapped to a vector). The hard question is how words are woven into sentences, that is, **word order**.

Bag-of-words models

- Discard order; treat text as an unordered *set* of tokens.
- Simple, fast, strong baselines (today).

Sequence models

- Process words in order, like a timeseries.
- RNNs and Transformers (later lectures).

We build bag-of-words models, then take the first step toward sequences with **word embeddings**.

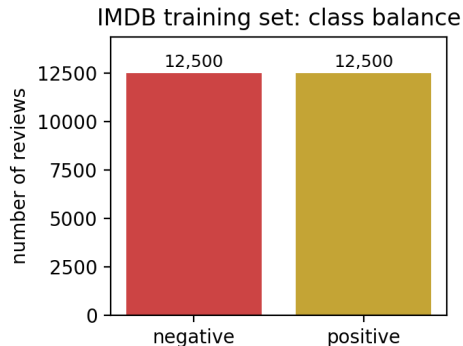


Table of Contents

3 Exploring the IMDB Data

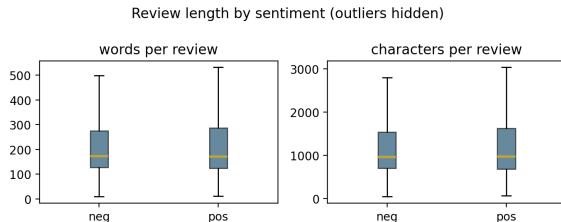
- ▶ Recap and Roadmap
- ▶ NLP: The Bird's-Eye View
- ▶ Exploring the IMDB Data
- ▶ Preparing Text Data
- ▶ Bag-of-Words on IMDB
- ▶ Word Embeddings
- ▶ Wrap-Up

- 50,000 movie reviews labeled **positive** or **negative**, split 25,000 train / 25,000 test, balanced exactly 12,500 / 12,500 per class.
- We use the *raw text* files, not the pre-vectorized version, to run the whole pipeline ourselves.
- Balance makes **accuracy a fair metric** and fixes the chance baseline at precisely **50%**.
- Best without external data: **~95%**.



Always look before you model. The next slides explore the data before we train anything.

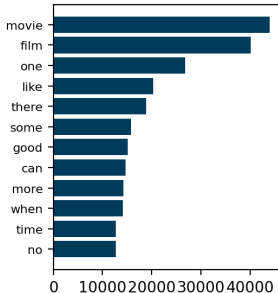
- Median review is ~ 175 words; most are under ~ 300 .
- The two classes have **near-identical** length profiles, so review length is *not* a sentiment giveaway.
- This justifies later capping sequences at `output_sequence_length = 600`: long enough to keep almost every review whole.



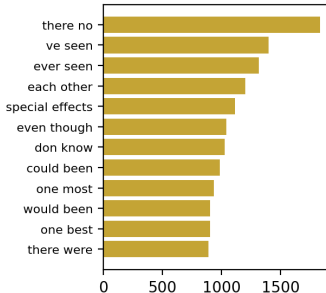
EDA 3: Frequent Unigrams, Bigrams, Trigrams

3 Exploring the IMDB Data

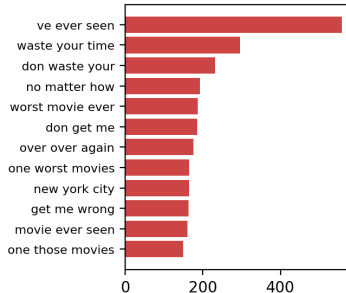
Top unigrams



Top bigrams



Top trigrams



- **Unigrams** (after dropping stop-words) are dominated by topic words: *movie, film, one*. Little polarity on their own.
- **Bigrams** start to carry sentiment and structure: *special effects, ever seen, each other*.
- **Trigrams** are sharply polar: *waste your time, worst movie ever, one worst movies*.

Why this matters

The signal sharpens as we widen the window. That is the empirical case for **n-grams**: local word order, which a plain unigram bag throws away, carries sentiment.



Table of Contents

4 Preparing Text Data

- ▶ Recap and Roadmap
- ▶ NLP: The Bird's-Eye View
- ▶ Exploring the IMDB Data
- ▶ **Preparing Text Data**
- ▶ Bag-of-Words on IMDB
- ▶ Word Embeddings
- ▶ Wrap-Up

Deep learning models are differentiable functions over tensors of numbers. They process **numeric tensors**, never raw text.

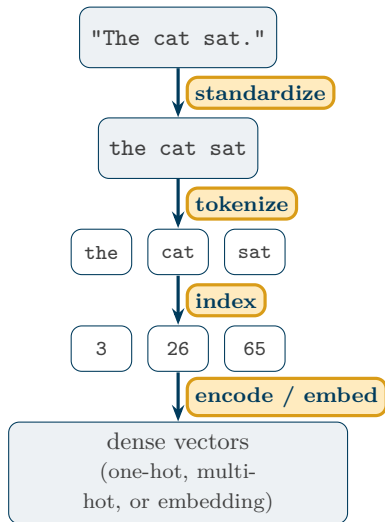
Definition: Vectorizing text

Mapping raw text to numeric tensors in three steps: **(1) standardize** the string, **(2) tokenize** it into units, **(3) index** each unit to an integer and then a vector.

The template is the same for every model; only the choices inside each step change.

The Vectorization Pipeline

4 Preparing Text Data



Two near-identical sentences can produce different byte strings: “México” vs. “Mexico”, “isn’t” vs. “isnt”, “I” vs. “i”.

- **Lowercase** and **strip punctuation** (the common default).
- Normalize accents (“é” → “e”); optionally **stem** (“staring”, “stared” → a shared root).

It cuts the data needed to generalize, but it can erase signal: a question detector should keep “?”. Standardization is a **choice**, not a default.

Definition: Token

The atomic unit a model vectorizes. Tokens may be **words**, **subwords**, or **characters**.

- **Word-level:** space/punctuation-separated substrings. Pairs with sequence models.
- **N-gram:** groups of N consecutive words (“the cat” is a bigram). Pairs with bag-of-words, where order is otherwise lost.
- **Character-level:** each character a token. Rare, specialized.

Extracting n-grams is manual feature engineering: a way to slip local word order back into an orderless model. Deep sequence models instead *learn* such groupings from raw token sequences.

Build an index of tokens from the **training data** and assign each a unique integer.

- Keep the top $\sim 20,000$ most frequent terms; rare terms add a huge, near-empty feature space.
- Reserved slots: index **0** = *mask* (padding), index **1** = *out-of-vocabulary* (OOV).
- Unknown tokens map to OOV, so unseen words never crash the lookup.

Batches must be rectangular, so shorter sequences are **padded** with the mask token to a common length.

Keras packages standardize, tokenize, and index into one fast layer over `tf.string` tensors:

```
1 from tensorflow.keras.layers import TextVectorization
2 text_vectorization = TextVectorization(output_mode="int")
3 text_vectorization.adapt(text_dataset)    # builds the vocabulary
```

- Defaults: lowercase + strip punctuation, split on whitespace (both overridable).
- Output modes: `int`, `multi_hot`, `count`, `tf_idf` (all used below).
- `get_vocabulary()` returns tokens sorted by frequency after slots 0 and 1, so “the” comes first.

Word-level indexing has a weakness: if training saw *low*, *new*, *newer* but never *lower*, the model has no representation for *lower* at test time. It collapses to the single OOV token.

Subword tokenization

Induce units *smaller* than words. Any unseen word becomes a **sequence of known subwords** (*lower* $\rightarrow$ *low* + *er*); in the worst case it falls back to characters, so nothing is truly out of vocabulary.

BPE is the standard subword algorithm behind modern language models. A **trainer** induces the vocabulary; an **encoder** applies it.

BPE training loop

1. Start with a vocabulary of all individual characters.
2. Find the most frequent adjacent pair of tokens in the corpus.
3. Merge that pair into one new token; add it to the vocabulary.
4. Repeat for k merges.

Merges run *within* words only (corpus pre-split on whitespace), so tokens never cross word boundaries.

Corpus token *renew* (with *new*, *reset* elsewhere). Each step merges the most frequent pair (highlighted in gold):

Step	Sequence	Merge
0	r e n e w	—
1	r e ne w	n e $\rightarrow$ ne
2	r e new	ne w $\rightarrow$ new
3	re new	r e $\rightarrow$ re
4	renew	re new $\rightarrow$ renew

The algorithm *discovers* the prefix *re-* and the word *new* from raw character statistics, with no linguistic rules supplied.

Why they win

- **No unknown words:** every string decomposes into known subwords; the OOV cliff disappears.
- **Controlled vocabulary:** k trades coverage against embedding-table size.
- **Morphology for free:** frequent affixes (*-ing*, *re-*) emerge as reusable tokens.
- **The modern default:** GPT, BERT, and relatives use BPE or close cousins.

A pragmatic default, not a theory of morphology. For today's demos we stay word-level, but production NLP is subword.

Where they struggle (merges follow frequency, not linguistics)

- **Rich morphology:** agglutinative (Turkish) and templatic (Arabic) words get shattered inconsistently, so one suffix splits many ways.
- **No-whitespace scripts:** Chinese, Japanese, Thai need a segmenter before BPE even starts.
- **Low-resource fairness gap:** English-trained merges encode other languages as more tokens: higher cost, less context.



Table of Contents

5 Bag-of-Words on IMDB

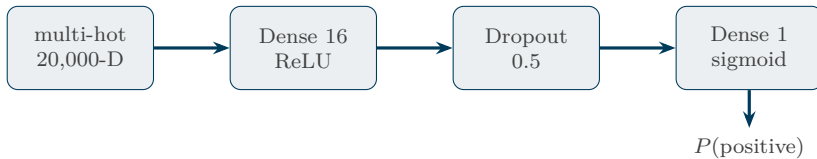
- ▶ Recap and Roadmap
- ▶ NLP: The Bird's-Eye View
- ▶ Exploring the IMDB Data
- ▶ Preparing Text Data
- ▶ **Bag-of-Words on IMDB**
- ▶ Word Embeddings
- ▶ Wrap-Up

Discard order; represent each review as the set of tokens it contains. “the cat sat on the mat” becomes

$$\{\text{cat, mat, on, sat, the}\}.$$

- Encode that set as one fixed-length vector over the 20,000-word vocabulary.
- **Binary (multi-hot):** a 1 for each present word, 0 elsewhere (sparse).
- However long the review, it collapses to a single vector, so a plain dense classifier suffices.

We change *only* the text representation, so any accuracy gap is attributable to the representation, not the model.



Loss: binary cross-entropy.
checkpoint.

Optimizer: RMSprop.

Best epoch restored by

```
1 def get_model(max_tokens=20000, hidden_dim=16):  
2     inputs = keras.Input(shape=(max_tokens,))  
3     x = layers.Dense(hidden_dim, activation="relu")(inputs)  
4     x = layers.Dropout(0.5)(x)  
5     outputs = layers.Dense(1, activation="sigmoid")(x)  
6     model = keras.Model(inputs, outputs)  
7     model.compile(optimizer="rmsprop",  
8                   loss="binary_crossentropy", metrics=["accuracy"])  
9     return model
```

The same `get_model()` is reused for all three bag-of-words representations below.

A plain unigram bag (multi-hot, 20,000-D) already scores **88.4%**, far above the 50% baseline: noting *which* words appear already captures most of the signal. But discarding order is crude: “not good” inverts “good”, and “United States” is not “united” plus “states”. **Bigrams** reinject local order as features (`ngrams=2`):

- “not good” becomes a single feature the model can weight negatively.
- The vocabulary now holds frequent unigrams *and* bigrams, still capped at 20,000.

Result — test accuracy: **89.6%**

A clear gain over the unigram model, just as the EDA’s polar trigrams suggested: order carries signal.

Instead of marking presence, **count** occurrences and reweight them so common words stop dominating.

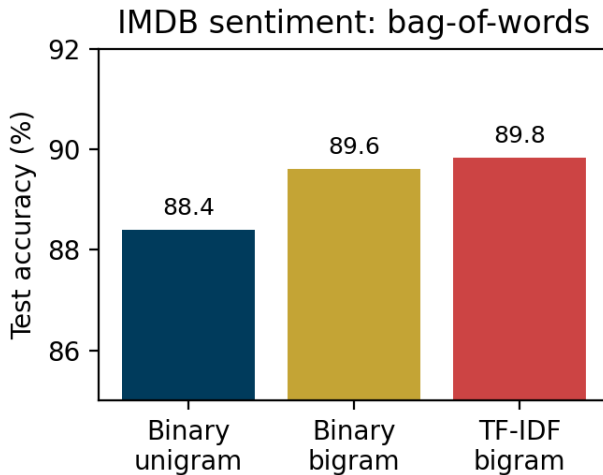
Definition: TF-IDF weight

$$\text{tf-idf}(t, d) = \underbrace{\text{tf}(t, d)}_{\text{count in this review}} \times \underbrace{\log \frac{N}{\text{df}(t)}}_{\text{rareness across all reviews}}$$

A term scores high when frequent *here* but rare *overall*. One switch:
`output_mode="tf_idf"`.

Result — test accuracy: **89.8%**

Essentially tied with the binary bigram (within run-to-run noise). The jump came from **bigrams**, not the reweighting. **Measure, don't assume.**



- **Bigrams matter most:** both bigram models (89.6%, 89.8%) clearly beat the unigram, so local word order is the single most useful refinement.
- **Unigram** (88.4%) is already strong, the reminder that simple baselines are hard to beat.
- **TF-IDF** (89.8%) and binary bigram are a statistical tie: the reweighting adds little once order is captured.
- All three sit below the $\sim 95\%$ ceiling: bag-of-words leaves signal on the table.

Fast, interpretable, excellent baselines, but they cannot represent that “brilliant” and “superb” are related words.

Accuracy is a fair score here *only* because IMDB is balanced. On a 95/5 split, a model that always predicts the majority class scores 95% while detecting nothing.

Definition: Precision, recall, F_1

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2PR}{P + R}$$

- **Precision:** of what we flagged positive, how much truly was?
- **Recall:** of all true positives, how many did we catch?
- F_1 : their harmonic mean, the usual single number when classes are imbalanced.

Report the metric that matches the **cost of each error type**, not just accuracy.

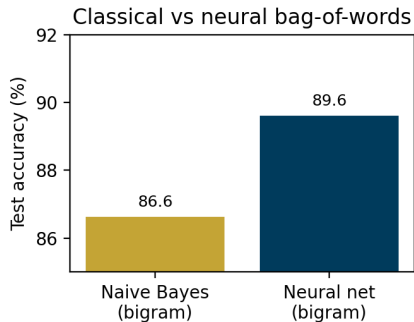
The classic bag-of-words classifier is **multinomial Naive Bayes**, a *generative* model: pick the class most likely to have produced the document.

Definition: Decision rule

$$\hat{c} = \arg \max_c P(c) \prod_i P(w_i | c),$$

words w_i assumed **conditionally independent** given the class.

- Independence is false (hence “naive”), yet works well; trained in a **single pass**.
- Our neural BoW head is *discriminative* (logistic regression): models $P(c | \text{text})$ directly.



Naive Bayes **86.6%** vs. neural bigram 89.6%.

A training-free model lands within a few points. **Earn your complexity.**

Strengths

- Fast to train, cheap to run.
- Strong, hard-to-beat baselines.
- Transparent: weights map to words.

Limits

- Huge, **sparse** vectors (20,000-D).
- Order mostly discarded.
- Every word is **independent**: no notion of similar meaning.

That last limit is the opening for **word embeddings**: a space where related words sit near each other.



Table of Contents

6 Word Embeddings

- ▶ Recap and Roadmap
- ▶ NLP: The Bird's-Eye View
- ▶ Exploring the IMDB Data
- ▶ Preparing Text Data
- ▶ Bag-of-Words on IMDB
- ▶ Word Embeddings
- ▶ Wrap-Up

A one-hot (or multi-hot) encoding makes every token **orthogonal** to every other: “movie” and “film” are as unrelated as “movie” and “Tuesday”. The vectors are **sparse**, **high-dimensional**, and **hardcoded**. Geometry should instead mirror meaning.

Definition: Word embedding

A dense, low-dimensional, *learned* vector per word, such that words used in similar contexts get nearby vectors.

	One-hot vectors	Word embeddings
Dimensionality	20,000+	100–1,024
Values	sparse (mostly 0)	dense floats
Origin	hardcoded	learned from data

Definition: Distributional hypothesis

“Words that occur in similar contexts tend to have similar meanings.”
(Joos 1950; **Harris** 1954; **Firth** 1957)

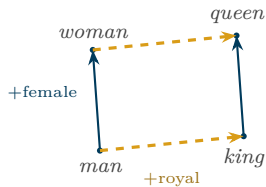
- Synonyms like *oculist* and *eye-doctor* appear near the same words (*eye*, *examined*).
- Difference in meaning corresponds roughly to difference in distribution.

This is the foundation of **vector semantics** and the first example of **representation learning**: learn meaning from the company a word keeps, rather than hand-coding it.

In a good space, specific **directions** are meaningful:

- A “gender” direction:
 $king - man + woman \approx queen$.
- A “plural” direction added to *king* lands near *kings*.

These regularities are not designed; they fall out of training on co-occurrence.



Parallel offsets encode consistent relations.

Definition: Cosine similarity

$$\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \|\mathbf{w}\|}$$

- **1** (same direction), **0** (orthogonal), **-1** (opposite).
- Dividing by lengths removes bias toward frequent (longer) word vectors.
- Measures **direction**, not magnitude.

Raw co-occurrence counts:

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry}, \text{info}) = 0.018$$

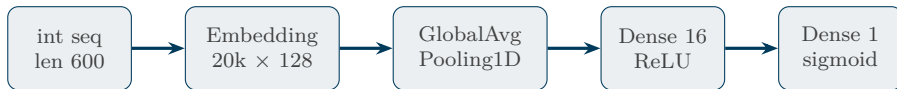
$$\cos(\text{digital}, \text{info}) = 0.996$$

information is far closer to *digital* than to *cherry*.

1. **Learn them jointly with your task.** Start from random vectors, train them by backprop alongside a classifier; the space specializes to your problem (e.g., movie sentiment).
2. **Use pretrained embeddings.** Load vectors precomputed on a large corpus (word2vec, GloVe) when task data is scarce, the text analogue of transfer learning.

Both run through the same Keras `Embedding` layer. We use option 1 on IMDB.

A learnable dictionary (integer $\rightarrow$ vector), pooled into one vector per review, then the same dense head.



```
1 embedded = layers.Embedding(max_tokens, 128, mask_zero=True)(inputs)
2 x = layers.GlobalAveragePooling1D()(embedded)
3 x = layers.Dense(16, activation="relu")(x)
4 outputs = layers.Dense(1, activation="sigmoid")(x)
```

`mask_zero=True` skips padding; weights start random and are **tuned by backprop**. We pull this matrix out afterward to inspect its geometry.

Trained from scratch on IMDB:

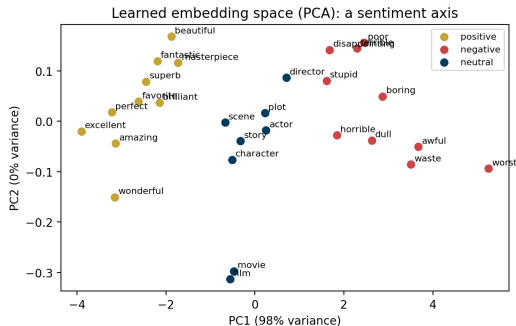
Test accuracy: **87.3%**

- Comparable to bag-of-words, but on dense 128-D vectors instead of 20,000-D sparse ones.
- The real payoff is not this number but the **structured space** learned along the way.

Pull out the trained embedding matrix and project curated words to 2-D with **PCA**:

- Positive and negative words separate cleanly along **PC1**; neutral words (*movie*, *plot*) sit in between.
- Trained jointly with sentiment and average-pooled, the space collapses onto a roughly **one-dimensional polarity axis**.

So the neighbors of “terrible” are *all* near-maximally similar (cosine ≈ 1): real but uninformative. General-purpose embeddings (next) train on **context prediction**, encoding far more than polarity.



word2vec: Learning Embeddings from Raw Text

6 Word Embeddings

How are general-purpose embeddings learned without labels? **word2vec** (Mikolov et al., 2013), specifically **skip-gram with negative sampling** (SGNS).

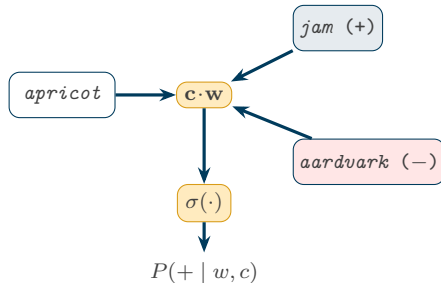
- Turn raw text into a **self-supervised** task: “Is word c likely to appear near target word w ?”
- Real context words are positives; randomly sampled words are negatives.
- We discard the prediction and keep the **learned weights** as the embeddings.

Running text is its own supervision signal, so no hand-labeling is needed.

Score a (target, context) pair by embedding similarity, squashed by the sigmoid:

$$P(+ \mid w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + e^{-\mathbf{c} \cdot \mathbf{w}}}$$

- High dot product $\Rightarrow$ high probability c is real context.
- **Negative sampling:** a few random non-context words per true pair.
- Training pulls true pairs together, pushes negatives apart.



- word2vec and **GloVe** (Stanford, 2014) are **static**: one fixed vector per word, regardless of sentence.
- So “mouse” (rodent) and “mouse” (device) collapse to the *same* vector.
- **Contextual** embeddings (the BERT family) give a word a *different* vector in each context, resolving the ambiguity.
- Contextual embeddings come from **Transformers**, which we build in upcoming lectures.

Static embeddings are the bridge from counting words to representing meaning, and the foundation for everything that follows.

The distributional hypothesis cuts both ways: a word's vector reflects *whatever* company it keeps in the corpus, human prejudice included.

Bias is learned, not designed

Word2vec and GloVe vectors reproduce analogies like *man : computer programmer :: woman : homemaker*, and tie gendered and racial terms to stereotyped occupations and sentiment (Bolukbasi et al. 2016; Caliskan et al. 2017).

- “Meaning from co-occurrence” also means **stereotypes from co-occurrence**.
- These vectors feed hiring, search, and moderation systems, so the bias propagates downstream.
- Mitigation (debiasing, balanced corpora, audits) is an active research area, not a solved problem.



Table of Contents

7 Wrap-Up

- ▶ Recap and Roadmap
- ▶ NLP: The Bird's-Eye View
- ▶ Exploring the IMDB Data
- ▶ Preparing Text Data
- ▶ Bag-of-Words on IMDB
- ▶ Word Embeddings
- ▶ Wrap-Up

1. **Explore first:** IMDB is balanced, the classes share a length profile, and frequent n-grams sharpen in polarity as the window widens.
2. Text is **standardized, tokenized, indexed** before any model sees it.
3. **Subword tokenization (BPE)** removes unknown words; the modern default for LLMs.
4. On one classifier, **bigrams** (89.6%) beat unigrams (88.4%); **TF-IDF** (89.8%) lands between, so word order helped most.
5. The classical **Naive Bayes** baseline (86.6%) trails the neural net by only a few points with no training: simple, generative, hard to beat.
6. **Word embeddings** give dense, learned vectors where the **distributional hypothesis** holds, measured by **cosine similarity**.
7. **word2vec / skip-gram** learns them self-supervised; **Transformers** will make them contextual.

Looking ahead

We can now turn words into meaningful vectors. Next we process them as **sequences**: recurrent networks that carry state across time, then the **attention** mechanism that replaced them, leading to the Transformer.

Reading for next session

- Chollet, Ch. 11, pp. 327–341 (sequence models and the Transformer encoder).
- Jurafsky & Martin, Ch. 6 (vector semantics and embeddings) for deeper word2vec detail.

Practice

- Run the companion notebook end to end; re-create the EDA plots and the PCA projection, then query the embedding space with your own words.